

Taller de Git!

ComCom

DC - FCEyN - UBA

September 11, 2026



Motivación

¿A qué vinieron?

- Herramientas necesarias para la vida diaria, tanto académica como profesional.

¿A qué vinieron?

- Herramientas necesarias para la vida diaria, tanto académica como profesional.
- No son obligatorias para la currícula, pero igualmente se esperan de nosotros.

¿A qué vinieron?

- Herramientas necesarias para la vida diaria, tanto académica como profesional.
- No son obligatorias para la currícula, pero igualmente se esperan de nosotros.
- Gastamos tiempo en tareas que deberían ser automatizables.

La Shell

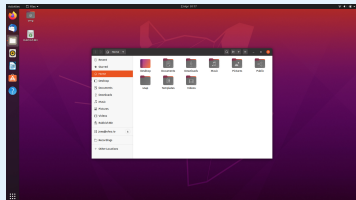
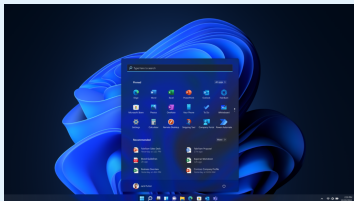
¿Qué es?

- Interactuamos con la computadora a través de una interfaz.

La Shell

¿Qué es?

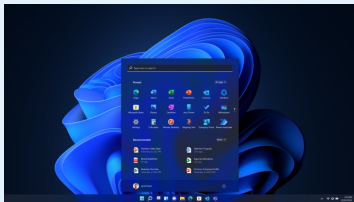
- Interactuamos con la computadora a través de una interfaz.
- Puede ser gráfica (GUI).



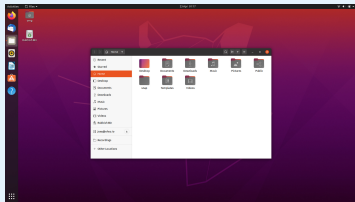
La Shell

¿Qué es?

- Interactuamos con la computadora a través de una interfaz.
- Puede ser gráfica (GUI).



- O de línea de comando (CLI).



La Shell (2)

¿Qué es? (2)

- Bash AKA *Bourne-Again SHell*

La Shell (2)

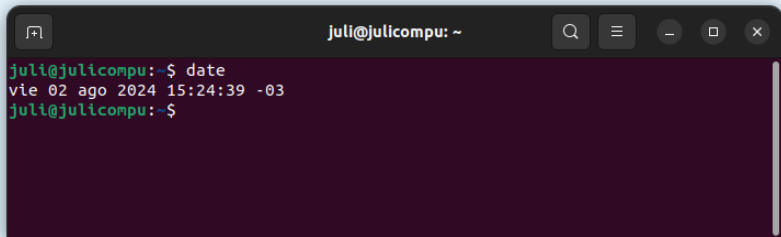
¿Qué es? (2)

- Bash AKA *Bourne-Again SHell*
- Para abrir un *shell prompt*, necesitamos una terminal.

La Shell (2)

¿Qué es? (2)

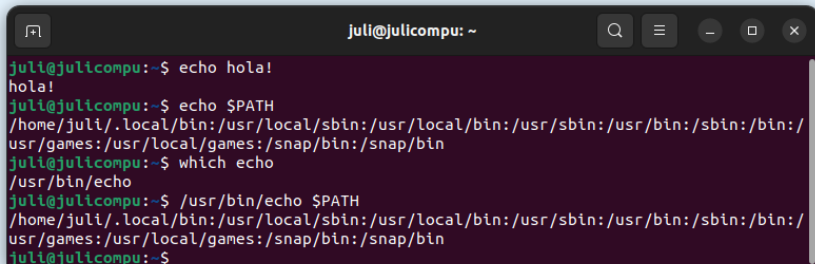
- Bash AKA *Bourne-Again SHell*
- Para abrir un *shell prompt*, necesitamos una terminal.

A terminal window with a dark background and light text. The title bar shows 'juli@julicompu: ~' and standard window controls. The terminal content shows the command 'date' being executed, resulting in the output 'vie 02 ago 2024 15:24:39 -03'. The prompt 'juli@julicompu:~\$' is visible at the end of each line.

```
juli@julicompu:~$ date
vie 02 ago 2024 15:24:39 -03
juli@julicompu:~$
```

La Shell (3)

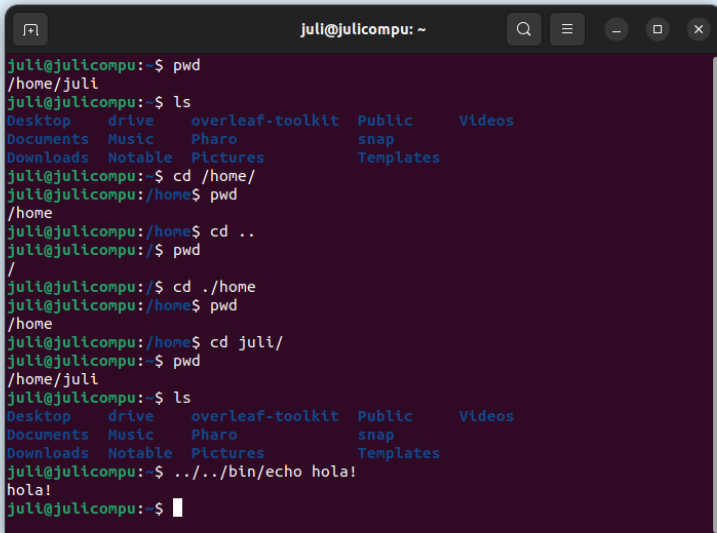
¿Cómo se usa?



```
juli@julichcompu: ~  
juli@julichcompu:~$ echo hola!  
hola!  
juli@julichcompu:~$ echo $PATH  
/home/juli/.local/bin:/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/games:/usr/local/games:/snap/bin:/snap/bin  
juli@julichcompu:~$ which echo  
/usr/bin/echo  
juli@julichcompu:~$ /usr/bin/echo $PATH  
/home/juli/.local/bin:/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/games:/usr/local/games:/snap/bin:/snap/bin  
juli@julichcompu:~$
```

La Shell (4)

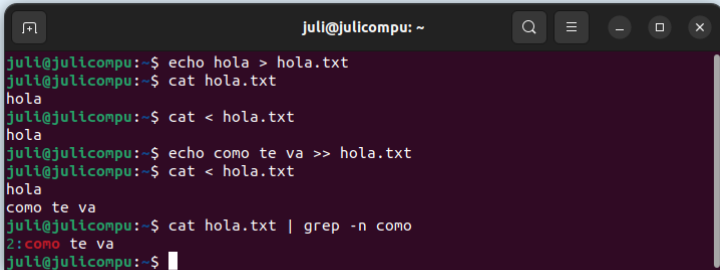
Navegando la Shell



```
juli@julichompu: ~  
juli@julichompu:~$ pwd  
/home/juli  
juli@julichompu:~$ ls  
Desktop  drive  overleaf-toolkit  Public  Videos  
Documents Music  Pharo             snap  
Downloads Notable Pictures          Templates  
juli@julichompu:~$ cd /home/  
juli@julichompu:/home$ pwd  
/home  
juli@julichompu:/home$ cd ..  
juli@julichompu:/ $ pwd  
/  
juli@julichompu:/ $ cd ./home  
juli@julichompu:/home$ pwd  
/home  
juli@julichompu:/home$ cd juli/  
juli@julichompu:~$ pwd  
/home/juli  
juli@julichompu:~$ ls  
Desktop  drive  overleaf-toolkit  Public  Videos  
Documents Music  Pharo             snap  
Downloads Notable Pictures          Templates  
juli@julichompu:~$ ../../bin/echo hola!  
hola!  
juli@julichompu:~$
```

La Shell (5)

Conectando programas



```
juli@julicompu: ~  
juli@julicompu:~$ echo hola > hola.txt  
juli@julicompu:~$ cat hola.txt  
hola  
juli@julicompu:~$ cat < hola.txt  
hola  
juli@julicompu:~$ echo como te va >> hola.txt  
juli@julicompu:~$ cat < hola.txt  
hola  
como te va  
juli@julicompu:~$ cat hola.txt | grep -n como  
2:como te va  
juli@julicompu:~$
```

Comandos útiles

Otros ejemplos

- touch
- curl
- nano
- unzip
- mkdir

Ejercicio

Ejecuten *man man*. ¿Qué sucedió? ¿Qué hace el programa *man*?
Úsenlo para averiguar los usos de los otros programas de más arriba.

Ejercitando los comandos

Ejercicio

- 1 Creen una carpeta para el taller, en donde ustedes quieran. Puede ser en el escritorio. Asegúrense de situarse dentro de ella.

Ejemplo: *mkdir carpeta*; *cd carpeta*

Recuerden que la carpeta se creará en la dirección donde estén parados en la terminal.

Ejercitando los comandos

Ejercicio

- 1 Creen una carpeta para el taller, en donde ustedes quieran. Puede ser en el escritorio. Asegúrense de situarse dentro de ella.

Ejemplo: *mkdir carpeta; cd carpeta*

Recuerden que la carpeta se creará en la dirección donde estén parados en la terminal.

- 2 Usen curl para bajar el archivo en bit.ly/taller-git-ej1.

Ejemplo: *curl -L "bit.ly/taller-git-ej1" > script.py*

Ejercitando los comandos

Ejercicio

- 1 Creen una carpeta para el taller, en donde ustedes quieran. Puede ser en el escritorio. Asegúrense de situarse dentro de ella.

Ejemplo: `mkdir carpeta`; `cd carpeta`

Recuerden que la carpeta se creará en la dirección donde estén parados en la terminal.

- 2 Usen curl para bajar el archivo en bit.ly/taller-git-ej1.

Ejemplo: `curl -L "bit.ly/taller-git-ej1" > script.py`

- 3 Usen cat para ver el contenido del archivo.

Ejemplo: `cat script.py`

Ejercitando los comandos

Ejercicio

- 1 Creen una carpeta para el taller, en donde ustedes quieran. Puede ser en el escritorio. Asegúrense de situarse dentro de ella.

Ejemplo: `mkdir carpeta`; `cd carpeta`

Recuerden que la carpeta se creará en la dirección donde estén parados en la terminal.

- 2 Usen curl para bajar el archivo en bit.ly/taller-git-ej1.

Ejemplo: `curl -L "bit.ly/taller-git-ej1" > script.py`

- 3 Usen cat para ver el contenido del archivo.

Ejemplo: `cat script.py`

- 4 Ejecuten el archivo haciendo `python3 script.py`

Ejercitando los comandos

Ejercicio

- 1 Creen una carpeta para el taller, en donde ustedes quieran. Puede ser en el escritorio. Asegúrense de situarse dentro de ella.

Ejemplo: `mkdir carpeta`; `cd carpeta`

Recuerden que la carpeta se creará en la dirección donde estén parados en la terminal.

- 2 Usen curl para bajar el archivo en bit.ly/taller-git-ej1.

Ejemplo: `curl -L "bit.ly/taller-git-ej1" > script.py`

- 3 Usen cat para ver el contenido del archivo.

Ejemplo: `cat script.py`

- 4 Ejecuten el archivo haciendo `python3 script.py`

- 5 Usando nano, cambien el comportamiento del programa para que imprima lo que ustedes quieran.

Ejemplo: `nano script.py`

Ejercitando los comandos

Ejercicio

- 1 Creen una carpeta para el taller, en donde ustedes quieran. Puede ser en el escritorio. Asegúrense de situarse dentro de ella.
Ejemplo: `mkdir carpeta`; `cd carpeta`
Recuerden que la carpeta se creará en la dirección donde estén parados en la terminal.
- 2 Usen curl para bajar el archivo en bit.ly/taller-git-ej1.
Ejemplo: `curl -L "bit.ly/taller-git-ej1" > script.py`
- 3 Usen cat para ver el contenido del archivo.
Ejemplo: `cat script.py`
- 4 Ejecuten el archivo haciendo `python3 script.py`
- 5 Usando nano, cambien el comportamiento del programa para que imprima lo que ustedes quieran.
Ejemplo: `nano script.py`
- 6 Vuelvan a ejecutar el archivo.

Motivación 1/2



tp_final.tex



tp_final_2.tex



tp_final_
este_va.tex



tp_final_
posta.tex



tp_final_
reentrega.tex

Motivación 1/2



tp_final.tex



tp_final_2.tex



tp_final_
este_va.tex



tp_final_
posta.tex



tp_final_
reentrega.tex



Motivación 2/2

Trabajando en grupo

- Enviar cambios por mail, o

Motivación 2/2

Trabajando en grupo

- Enviar cambios por mail, o
- Enviar archivos por Discord, o

Motivación 2/2

Trabajando en grupo

- Enviar cambios por mail, o
- Enviar archivos por Discord, o
- Sincronizar cambios por Google Docs.

Motivación 2/2

Trabajando en grupo

- Enviar cambios por mail, o
- Enviar archivos por Discord, o
- Sincronizar cambios por Google Docs.

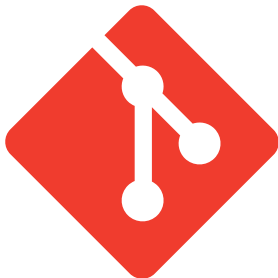


¿Qué es un Sistema de Control de Versiones?

- 1 Programas que permiten **manejar los cambios** en el código fuente de un proyecto a lo largo del tiempo.
- 2 Llevan un **seguimiento** de las modificaciones que hacemos, y en caso de que nos equivoquemos, es posible volver atrás y comparar el código actual con versiones anteriores para ayudar a arreglar el error.
- 3 Permiten que distintas personas modifiquen el código a la vez y **compartan los cambios**.

¿Qué es Git? 1/3

- Sistema de Control de Versiones **distribuido y de código abierto**.
- Con énfasis en la **performance** (para manejar proyectos muy grandes), **seguridad** y **flexibilidad**.
- Amplio conjunto de comandos que permiten realizar operaciones de alto y bajo nivel.
- Mantiene una copia local completa del proyecto.



¿Qué es Git? 2/3

GIT - the stupid content tracker

"git" can mean anything, depending on your mood.

- **random three-letter combination that is pronounceable**, and not actually used by any common UNIX command. The fact that it is a mispronunciation of "get" may or may not be relevant.
- **stupid**. contemptible and despicable. simple. Take your pick from the dictionary of slang.
- **"global information tracker"**: you're in a good mood, and it actually works for you. Angels sing, and a light suddenly fills the room.
- **"goddamn idiotic truckload of sh*t"**: when it breaks

¿Qué es Git? 2/3

GIT - the stupid content tracker

"git" can mean anything, depending on your mood.

- **random three-letter combination that is pronounceable**, and not actually used by any common UNIX command. The fact that it is a mispronunciation of "get" may or may not be relevant.
- **stupid**. contemptible and despicable. simple. Take your pick from the dictionary of slang.
- **"global information tracker"**: you're in a good mood, and it actually works for you. Angels sing, and a light suddenly fills the room.
- **"goddamn idiotic truckload of sh*t"**: when it breaks

- Mensaje del commit de Linus Torvalds agregando el README al repositorio Git de Git (uf)

¿Qué es Git? 3/3

¿Y entonces?

- Es un programa que nos permite hacer un seguimiento del estado de un **repositorio**.

¿Qué es Git? 3/3

¿Y entonces?

- Es un programa que nos permite hacer un seguimiento del estado de un **repositorio**.
- Se puede usar de manera local, o subir un **repositorio** a internet y usarlo de manera colaborativa.

¿Qué es Git? 3/3

¿Y entonces?

- Es un programa que nos permite hacer un seguimiento del estado de un **repositorio**.
- Se puede usar de manera local, o subir un **repositorio** a internet y usarlo de manera colaborativa.

¿Repositorio? ¿Y eso?

¿Qué es Git? 3/3

¿Y entonces?

- Es un programa que nos permite hacer un seguimiento del estado de un **repositorio**.
- Se puede usar de manera local, o subir un **repositorio** a internet y usarlo de manera colaborativa.

¿Repositorio? ¿Y eso?

- La unidad básica donde guardaremos todos los elementos de nuestro proyecto.

¿Qué es Git? 3/3

¿Y entonces?

- Es un programa que nos permite hacer un seguimiento del estado de un **repositorio**.
- Se puede usar de manera local, o subir un **repositorio** a internet y usarlo de manera colaborativa.

¿Repositorio? ¿Y eso?

- La unidad básica donde guardaremos todos los elementos de nuestro proyecto.
- Es un directorio o carpeta.

¿Qué es Git? 3/3

¿Y entonces?

- Es un programa que nos permite hacer un seguimiento del estado de un **repositorio**.
- Se puede usar de manera local, o subir un **repositorio** a internet y usarlo de manera colaborativa.

¿Repositorio? ¿Y eso?

- La unidad básica donde guardaremos todos los elementos de nuestro proyecto.
- Es un directorio o carpeta.
- La idea es que nuestro proyecto, y todos los archivos que lo contengan, existan dentro de este **repositorio**.

¿Qué es Git? 3/3

¿Y entonces?

- Es un programa que nos permite hacer un seguimiento del estado de un **repositorio**.
- Se puede usar de manera local, o subir un **repositorio** a internet y usarlo de manera colaborativa.

¿Repositorio? ¿Y eso?

- La unidad básica donde guardaremos todos los elementos de nuestro proyecto.
- Es un directorio o carpeta.
- La idea es que nuestro proyecto, y todos los archivos que lo contengan, existan dentro de este **repositorio**.
- Sabemos que una carpeta es un **repositorio** de Git porque tiene dentro una subcarpeta *.git*.

Configuraciones iniciales

Tu identidad

Es importante establecer nuestro **nombre y email** en nuestro repositorio, ya que estos van a ir asociados con los cambios que hagamos:

```
git config --global user.name "Guybrush Threepwood"  
git config --global user.email guybrush@example.com
```

También se puede omitir el flag `--global` para que las credenciales apliquen solo al repositorio en el que estamos trabajando.

Creando un repositorio

```
git init
```

Creando un repositorio

git init

Crea un repositorio local.

- 1 Nos paramos en el directorio que queremos convertir en un repositorio.
- 2 Ejecutamos `git init`.

Esto crea un subdirectorio `.git` que tiene todos los archivos necesarios de Git.

Creando un repositorio

git init

Crea un repositorio local.

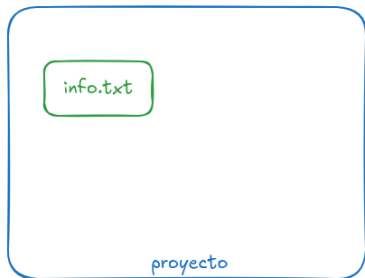
- 1 Nos paramos en el directorio que queremos convertir en un repositorio.
- 2 Ejecutamos `git init`.

Esto crea un subdirectorio `.git` que tiene todos los archivos necesarios de Git.

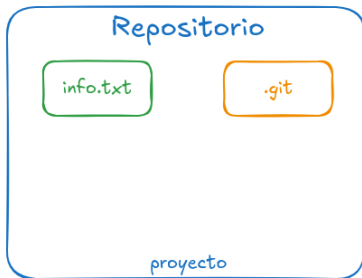
Ejercicio

Creen un directorio o carpeta nueva. Dentro de ella, ejecuten `git init`. Recuerden los comandos de bash `cd`, `ls`, `mkdir`.

¿Directorio o repositorio?



Directorio 'proyecto' **antes** de hacer *git init*



Directorio 'proyecto' **luego** de hacer *git init*

¿Está preparado, confirmado o ninguna de las dos?

git status

Output de ejemplo

```
nothing to commit (create/copy files and use "git add" to  
track)
```

¿Está preparado, confirmado o ninguna de las dos?

git status

Output de ejemplo

```
nothing to commit (create/copy files and use "git add" to  
track)
```

Ejercicio

Crear un nuevo archivo en nuestro repositorio y ejecuten *git status*.
Vean como cambia el mensaje. Recuerden el comando *touch*.

¿Está preparado, confirmado o ninguna de las dos?

git status

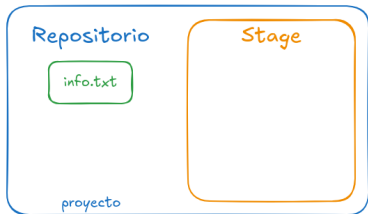
Output de ejemplo

```
Untracked files: (use "git add <file>..." to include in
                  what will be committed)
                  archivo
```

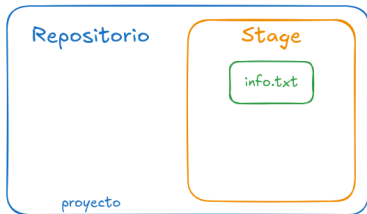
Ejercicio

Ejecuten el comando que les sugiere *git* para incluir el archivo que crearon recién. Hagan *git status*.

¿Está preparado, confirmado o ninguna de las dos?



Directorio 'proyecto' **antes** de hacer *git add info.txt*



Directorio 'proyecto' **luego** de hacer *git add info.txt*

¿Está preparado, confirmado o ninguna de las dos?

git status

Output de ejemplo

```
Changes to be committed: (use "git rm --cached <file>..."  
                        to unstage)  
    new file:   archivo
```

Con esto vemos que *git* ahora conoce al nuevo archivo, y lo va a tener en cuenta cuando hagamos un *commit* próximamente.

¿Está preparado, confirmado o ninguna de las dos?

git status

Output de ejemplo

```
Changes to be committed: (use "git rm --cached <file>..."  
                        to unstage)  
    new file:   archivo
```

Con esto vemos que *git* ahora conoce al nuevo archivo, y lo va a tener en cuenta cuando hagamos un *commit* próximamente.

Confirmando cambios

```
git commit
```

Confirmando cambios

git commit

Una vez que tenemos ciertos cambios marcados con *add*, podemos confirmarlos ejecutando `git commit -m "mensaje"`.

Donde [mensaje] es una breve descripción de los cambios que acabamos de confirmar.

Confirmando cambios

git commit

Una vez que tenemos ciertos cambios marcados con *add*, podemos confirmarlos ejecutando `git commit -m "mensaje"`.

Donde [mensaje] es una breve descripción de los cambios que acabamos de confirmar.

¡Yo elijo lo que pongo en el stage!

- Puedo poner y sacar archivos como *staged* usando *git add* y *git rm -cached*.
- Git nos sugiere estos comandos luego de hacer *git status*.

Confirmando cambios

git commit

Una vez que tenemos ciertos cambios marcados con *add*, podemos confirmarlos ejecutando `git commit -m "mensaje"`.

Donde [mensaje] es una breve descripción de los cambios que acabamos de confirmar.

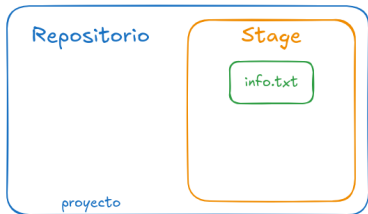
¡Yo elijo lo que pongo en el stage!

- Puedo poner y sacar archivos como *staged* usando *git add* y *git rm -cached*.
- Git nos sugiere estos comandos luego de hacer *git status*.

Ejercicio

Hagan el *commit* que venían preparando y elijan un mensaje. Luego hagan *git status*.

¿Está preparado, confirmado o ninguna de las dos?



Directorio 'proyecto' **antes** de hacer *commit*



Directorio 'proyecto' **luego** de hacer *git commit -m "nuevo info.txt"*

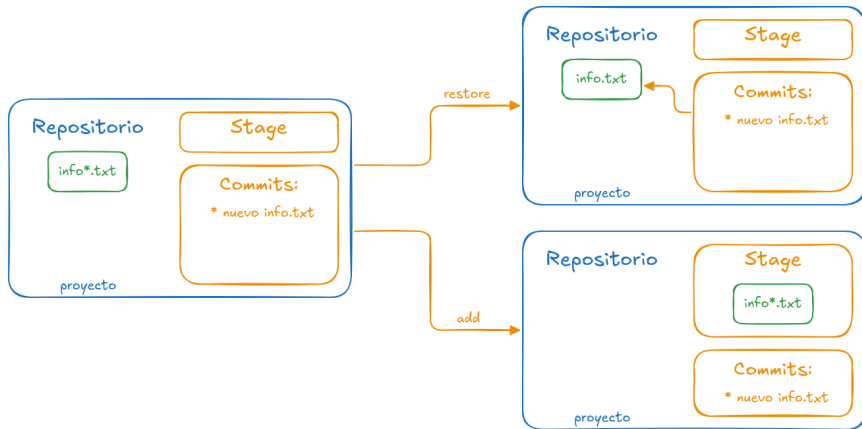
¿Y si modifico el archivo luego del commit?

git status

Output de ejemplo

```
Changes not staged for commit:
(use "git add <file>..." to update what will be
    committed)
(use "git restore <file>..." to discard changes in
    working directory)
modified: archivo
```

Elige tu propia aventura

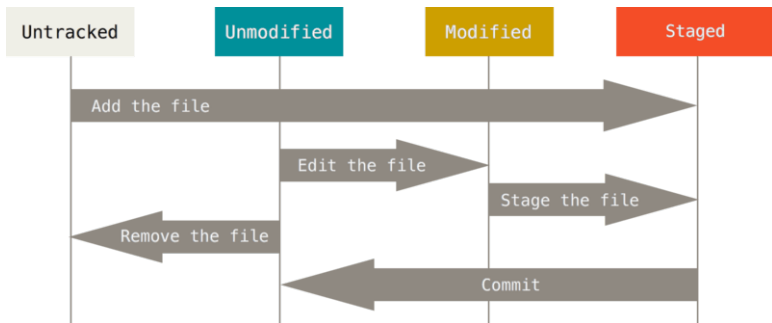


Nosotros decidimos que hacer con los nuevos cambios. Los añadimos al stage con *add*, o los rechazamos con *restore*. Tener en cuenta que al rechazar los cambios, volvemos al estado del archivo en el último commit.

Estados de un archivo

Los cuatro estados de los archivos

- 1 Untracked = Sin seguimiento
- 2 Unmodified = Sin modificar
- 3 Modified = Modificado
- 4 Staged, to be committed = Confirmado



git log

Historial

git log

Este comando nos sirve para ver el historial de cambios, o commits. Además nos dice quién hizo cada cosa, cuando, y su correo electrónico.

git log

Este comando nos sirve para ver el historial de cambios, o commits. Además nos dice quién hizo cada cosa, cuando, y su correo electrónico.

	COMMENT	DATE
○	CREATED MAIN LOOP & TIMING CONTROL	14 HOURS AGO
○	ENABLED CONFIG FILE PARSING	9 HOURS AGO
○	MISC BUGFIXES	5 HOURS AGO
○	CODE ADDITIONS/EDITS	4 HOURS AGO
○	MORE CODE	4 HOURS AGO
○	HERE HAVE CODE	4 HOURS AGO
○	AAAAAAA	3 HOURS AGO
○	ADKFJSLKDFJSDKLFJ	3 HOURS AGO
○	MY HANDS ARE TYPING WORDS	2 HOURS AGO
○	HAAAAAAAANDS	2 HOURS AGO

AS A PROJECT DRAGS ON, MY GIT COMMIT
MESSAGES GET LESS AND LESS INFORMATIVE.

Fuente: <https://xkcd.com/1296/>

Bibliografía

- MiT missing semester: <https://missing.csail.mit.edu/>
- Documentación de Git: <https://git-scm.com/docs/user-manual>